# 项目申报表

青年开源专项基金种子计划 · 第二批

## 一、项目基本情况

| | |
|---|---|
| **项目名称** | SwarmBox（多智能体不可逃逸测试床） |
| **项目地址** | `https://github.com/Zhiyilang/swarmbox` |
| **国内镜像** | `https://gitee.com/Zhiyilang/swarmbox` |
| **开源许可证** | **MIT**（OSI 认可的开源许可） |
| **所属方向** | AI Agent 基础设施 / AI Infra |
| **开发语言** | Python 3.9+ |
| **第三方依赖** | **无**（纯标准库），零网络访问 |
| **项目联系人** | 支一郎（独立开发者，唯一维护者） |
| **当前规模** | 11 个核心模块 + 235 条自动化断言 |

## 二、项目简介

SwarmBox 是一个**不可逃逸、可复跑的多智能体测试床**，为 AI Agent 的安全与对齐研究而设计。

它最根本的设计选择是：**不解释执行智能体的输出，而是把输出当作纯数据来解析。**因此文件、网络、进程、import、反射等逃逸手段根本不在语法里——不是"被禁止"，而是"无从调用"。这使"不可逃逸"成为一个结构性事实，而不是一份需要不断维护的禁止清单。

每次提案都要经过三重校验：结构（`check_shape`）、身份（`check_identity`）、状态（`check_state`）；任一不过即被闸门 `screen()` 拒绝，并写入只追加账本（append-only ledger）。因此"从未发生逃逸"是可审计的历史记录，而非一句需要被相信的承诺。

项目内置公共资源池经济，可观测 5–20 个智能体在共享资源下是协作、合谋还是耗尽资源。复现成本极低：`git clone && python eval.py`，离线运行 235 条断言，0 失败。

## 三、要解决的问题

多智能体系统正从实验室走向生产环境。一旦给一群智能体授予工具与共享资源，风险就从"单个模型会不会乱说"升级为"一群智能体会不会合谋、逃逸、耗尽资源"——而这在上线前几乎无法系统性检验。当前存在三个具体缺口：

- **逃逸风险无法预检**：智能体被授予工具权限后，文件 / 网络 / 进程等逃逸路径难以穷举验证。
- **禁止清单式沙箱易被绕过**：靠"不许调用 X"实现的隔离总会有遗漏，且无法证明已穷尽。
- **群体行为缺乏实验环境**：合谋、资源耗尽等现象缺少可复现、可量化的观测平台。

SwarmBox 把这三个问题各自落地为可运行、可断言的代码，而不是停留在方案描述层面。

## 四、技术方案

| | |
|---|---|
| **世界模型 `world.py`** | `class World` 维护公共资源池与系统不变量，违反时抛 `InvariantError` |
| **闸门 `gate.py`** | `screen()` 串联 `check_shape()` / `check_identity()` / `check_state()` 三重校验 |
| **账本 `ledger.py`** | `class Ledger` 只追加记录全部被拒提案，支持 `read_ledger()` 回放 |
| **攻击语料 `probe.py`** | 32 条逃逸尝试，用于验证不可逃逸性 |
| **保真度 `fidelity.py`** | `aggregate()` / `compare()` 支持多规模多轮次消融实验 |
| **其余模块** | `agents.py`（智能体）、`swarm.py`（群体调度）、`metrics.py`（指标）、`chain.py`（链式实验） |

**关键实现要点**：智能体输出全程作为数据结构处理，从不进入任何执行或求值路径；所有阈值与规则属于世界配置，**不由提案内容决定**，因此智能体无法用输出影响自身约束。

## 五、创新点

- **不解释执行，只作数据解析**——逃逸手段不在语法里，攻击面不存在，而非被封堵。
- **三重校验层层收窄**——结构、身份、状态三层各自独立，任一不过即拒绝。
- **可审计而非需信任**——只追加账本使安全性成为可核对的历史记录。
- **群体行为可量化**——合谋相关系数、collapse_rate、gini 等指标把"群体是否合谋"变成可计算的量。
- **零依赖零网络**——纯 Python 标准库，任何环境一次命令即可完整复现。

## 六、实测佐证（本机实跑，可复现）

| 命令 | 实测结果 |
|---|---|
| `python eval.py` | **`235 passed, 0 failed`** |
| `python -m swarmbox gate` | `32` 条攻击语料，`31` 条命中预期规则 `ID`，**`escape successes 0`** |
| `python -m swarmbox run` | `invariants` **`0 violations`**；`collusion corr −0.831`；`covert channel 10 cross-agent reads` |
| `python -m swarmbox fidelity` | `collapse_rate 1.0000`；`gini 0.1649`；`escape 0`；`violations 0` |

## 七、开源与社区情况

- GitHub、Gitee **双平台公开**，MIT 许可，任何人可自由使用、修改、分发。
- 提供中英双语 README，附 REPORT.pdf 技术报告与 79 秒演示视频（由真实命令输出自动生成）。
- 零依赖设计使贡献者无需配置任何环境即可参与，`git clone` 后一条命令验证。

## 八、后续发展计划

- **扩展逃逸语料库**：从 32 条扩展到 100+ 条，覆盖同形字、归一化绕过、结构膨胀等新型手法。
- **增加资源策略维度**：引入配额、优先级、惩罚机制，观察更丰富的群体动力学。
- **可视化回放工具**：把只追加账本渲染成时间轴，直观呈现"谁在何时试图逃逸、命中哪条规则"。
- **对齐实验基线**：发布标准对照实验配置，供其他研究者做可比较的合谋 / 资源耗尽研究。

- **文档与教程**：补充中文教程，降低 AI 安全研究者与本专科学生的上手门槛。

## 九、维护者情况

独立开发者，具备 Python 工程与自动化（RPA）开发经验，可独立持续维护本项目。项目采用零第三方依赖设计，长期维护成本低，不存在因上游依赖弃坑而导致的不可维护风险。

## 十、资金用途（如获选）

- **服务器与测试资源**：用于大规模（20+ 智能体、长轮次）群体实验的算力。
- **文档与教程建设**：中文文档、教学示例、可视化工具。
- **社区运营**：逃逸语料库众包维护与评审。